

Спецификация требований к программному обеспечению (SRS)

Симулятор работы лифтов в здании

Студент: Недосекин Александр Сергеевич

Группа: ЭФБО 15-24

Дисциплина: Программирование корпоративных систем

2025–2026 уч. г.

1. Введение

1.1. Цель

Данный документ описывает требования к программному обеспечению "Симулятор работы лифтов в здании" — многопоточной системе моделирования работы группы лифтов, обрабатывающих вызовы с этажей и из кабин.

Документ предназначен для разработчиков, преподавателей и заказчиков учебного проекта.

1.2. Область действия (Scope)

В scope:

- Моделирование здания с настраиваемым числом этажей и лифтов
- Многопоточная работа лифтов и диспетчера
- Обработка вызовов с этажей и из кабины
- Консольный интерфейс (интерактивный и демо-режим)

Вне scope:

- Графический 3D-интерфейс
- Интеграция с реальным оборудованием лифтов
- Сетевая многопользовательская работа
- Сохранение истории в базу данных

1.3. Определения и аббревиатуры

Термин	Определение
SRS	Software Requirements Specification — спецификация требований
Hall Call	Вызов лифта с этажа (кнопка "вверх" или "вниз")
Cabin Call	Вызов этажа из кабины лифта
Dispatcher	Диспетчер — компонент, назначающий вызовы лифтам
Elevator	Лифт — исполнитель, перемещающийся между этажами
SCAN	Алгоритм обслуживания вызовов в текущем направлении движения

2. Общее описание

2.1. Описание продукта

Продукт — консольное приложение на C++, имитирующее работу лифтовой системы в многоэтажном здании. Каждый лифт работает в отдельном потоке, диспетчер в фоновом потоке распределяет входящие вызовы. Пользователь наблюдает состояние лифтов и может создавать вызовы вручную или через демо-режим.

2.2. Функции продукта

- Инициализация здания и лифтов
- Приём и очередь вызовов
- Диспетчеризация вызовов по эвристическому алгоритму
- Движение лифтов, остановка и имитация открытия дверей
- Отображение статуса в консоли
- Интерактивное управление и автоматическая демонстрация

2.3. Характеристики пользователей

Роль	Описание	Навыки
Оператор/студент	Запускает симуляцию, вводит вызовы	Базовые навыки работы с консолью
Разработчик	Настраивает параметры, изучает алгоритмы	C++, многопоточность
Преподаватель	Проверяет соответствие ТЗ	Понимание UML и SRS

3. Детальные требования

3.1. Методология сбора требований

- 1. Анализ предметной области — изучение принципов работы лифтовых систем (hall call, cabin call, диспетчеризация).
- 2. User Stories — формулировка потребностей пассажира, диспетчера и разработчика.
- 3. Story Mapping — выделение сценариев: "вызов с этажа", "поездка в кабине", "наблюдение за системой".
- 4. Иерархия Вигерса — классификация требований на бизнес-, пользовательские, функциональные и нефункциональные (разделы 3.2-3.5).

3.2. Бизнес-требования

ID	Требование
BR-01	Снизить время ожидания лифта за счёт оптимального распределения вызовов
BR-02	Обеспечить безопасную имитацию без риска для людей и оборудования
BR-03	Дать инженерам возможность тестировать алгоритмы диспетчеризации до внедрения в реальное здание
BR-04	Поддерживать здания с переменным числом этажей и лифтов

3.3. Пользовательские требования

ID	Требование
UR-01	Пользователь может вызвать лифт "вверх" с любого этажа, кроме верхнего
UR-02	Пользователь может вызвать лифт "вниз" с любого этажа, кроме нижнего
UR-03	Пассажир в кабине может задать этаж назначения
UR-04	Оператор видит текущее состояние всех лифтов в реальном времени
UR-05	Оператор может запустить автоматическую демонстрацию работы системы

3.4. Функциональные требования

ID	Требование
FR-01	Система создаёт здание с N этажами ($N \geq 2$) и M лифтами ($M \geq 1$)
FR-02	Система принимает вызовы с этажей (Hall Up / Hall Down)
FR-03	Система принимает вызовы из кабины (Cabin)
FR-04	Диспетчер назначает вызов наиболее подходящему лифту
FR-05	Лифт перемещается между этажами с моделированием задержки
FR-06	Лифт открывает двери на этажах назначения
FR-07	Каждый лифт выполняется в отдельном потоке
FR-08	Главный цикл диспетчеризации работает в отдельном потоке
FR-09	Консольный интерфейс принимает команды пользователя
FR-10	Демо-режим генерирует случайные вызовы в течение заданного времени

3.5. Нефункциональные требования

ID	Требование
NFR-01	Язык реализации — C++17
NFR-02	Использование стандартной библиотеки потоков (std::thread, мьютексы)
NFR-03	Время отклика обновления состояния — не более 1 с
NFR-04	Корректное завершение всех потоков при выходе из программы
NFR-05	Сборка через CMake (Windows, MinGW)
NFR-06	Исходный код структурирован по модулям (Building, Elevator, Dispatcher, Simulator)

4. Диаграммы UML

4.1. Диаграмма вариантов использования (Use Case)

Диаграмма (Mermaid, см. также docs/uml/*.puml):

```
flowchart LR
```

```
subgraph Ac
```

Акторы: Пассажир, Оператор.

Прецеденты: вызвать лифт вверх/вниз, выбрать этаж в кабине, просмотреть статус, запустить демо, диспетчеризовать вызовы, перемещать лифт.

Связи: вызовы включают диспетчеризацию; диспетчеризация включает перемещение лифта.

4.2. Диаграмма классов (Class Diagram)

Диаграмма (Mermaid, см. также docs/uml/*.puml):

```
classDiagram
```

```
class Simula
```

Классы: Simulator, Building, Elevator, Dispatcher, CallRequest, Direction.

Связи: Simulator содержит Building и Dispatcher; Building содержит несколько Elevator и очередь CallRequest; Dispatcher назначает вызовы лифтам.

4.3. Диаграмма последовательности — "Вызов лифта с этажа"

Диаграмма (Mermaid, см. также docs/uml/*.puml):

```
sequenceDiagram
```

```
actor User as
```

Сценарий: пассажир вводит и 5 → Simulator передаёт вызов в Building → Dispatcher забирает вызов из очереди, выбирает лифт → лифт едет на 5-й этаж, открывает двери → статус обновляется в консоли.

5. Архитектура реализации

Модуль	Файлы	Ответственность
types	include/types.hpp	Перечисления, структура CallRequest
Building	building.hpp/cpp	Здание, очередь вызовов, управление лифтами
Elevator	elevator.hpp/cpp	Поток лифта, движение, двери
Dispatcher	dispatcher.hpp/cpp	Выбор лифта для вызова
Simulator	simulator.hpp/cpp	UI, демо, главный цикл
main	src/main.cpp	Точка входа, разбор аргументов

6. Приложение — PlantUML

Исходники диаграмм: docs/uml/use_case.puml, class_diagram.puml, sequence_diagram.puml.